PRODUCT AND DESIGN BRIEF

NEIGHBORHOOD OS

A civic community platform where every neighborhood gets a live isometric map. Residents report problems, neighbors confirm them, coordinators verify and assign them, volunteers fix them, and proof of the fix goes back on the map.

This document describes what the app does, every page and panel, the roles that see them, the data each screen needs, and every API endpoint. It was generated by scanning the codebase (React + Vite + TypeScript frontend, Express server, Supabase Postgres) and is meant as the input for redesigning the screens in Claude Design.

Frontend	React 18, Vite, TypeScript, Framer Motion, Recharts, lucide-react icons, custom SVG/canvas isometric map
Backend	Express (`server/index.ts`), Postgres via Supabase, optional Claude API, Twilio, web push, SMTP
Languages	English and Hindi (India focus)
Modes	Simulated demo (no account, runs in browser) and Real neighborhoods (Supabase sign-in)

1. What the product does

The core loop: Report, then Confirm, then Verify and Assign, then Fix, then Prove it on the map.

1. **Report.** A resident clicks the map, writes (or speaks, in Hindi or English) a short description, picks a category and priority, optionally adds a photo. The app flags possible duplicates nearby.
2. **Confirm.** Neighbors press "I see this too" to confirm. Three neighbor flags hide a report until a coordinator reviews it.
3. **Verify and assign.** Only a human coordinator or moderator can mark a report "Verified". A coordinator approves an action plan, which creates a community mission, and assigns volunteers.
4. **Fix.** Volunteers join missions and post progress updates and evidence photos.
5. **Prove.** A coordinator asks for resolution review, a different moderator approves it (four-eyes rule), and a before/after photo slider is shown on the issue. The map pin turns green.

Feature groups

Group	What it does
Live isometric map	Real OpenStreetMap roads, buildings, parks and water drawn as an isometric city. Alternate Flat and List views. Status filters, resource toggle, priority heatmap, volunteer overlay, "Then vs now" time slider, map-picking mode for locations.
Issue reporting and lifecycle	Categories: pothole, garbage, streetlight, water, unsafe, blocked road, other. Priorities: low to critical. Lifecycle: Reported, Under review, Assigned, In progress, Resolution pending review, Resolved. Visible 0-100 priority score with the reasons listed. Duplicate detection. Spam, withdraw, hide and restore.
AI agent workflow	Understands a report (language, category, priority, English summary), picks the complaint route and authority, drafts a complaint in English or Hindi, waits for human approval, tracks reference numbers, prepares escalations on deadline, suggests a mission. Never sends anything automatically. Rule-based fallback when there is no API key.
Missions and volunteers	Community missions with progress bar, volunteer profile (availability, skills, emergency abilities), optional approximate-area sharing, suggested volunteers for an issue.
Resource radar	Shelters, water points, aid posts, donation drops, assembly points, equipment. Delivery notes and shortage flags. Neighbor exchange (offers and requests with a message thread).
World layer and Crisis Mode	Global map of official hazard feeds (USGS, NASA EONET, GDACS, local CAP feeds). Hazards that overlap a neighborhood raise an alert level: watch shows a heads-up, warning or severe switches the neighborhood into Crisis Mode (red alert chip, official banner, prominent SOS tab).
SOS (consent-based)	Request help with explicit consent and a chosen location. Visible only to the author, coordinators and an assigned volunteer. Location wiped on close or after 24 h. One active request per person, 3 per day. Always tells the user to call local emergency services first.
Safety and preparedness	Coordinator-started safety check-ins ("I am safe" / "I need help"), optional extra-help register, preparedness plan (routes, what to keep ready, contacts), notice board with pinned and urgent notices.
Reporting to authorities	Council reports with filters, weekly trends, hotspots, hand-off tracker (acknowledged / overdue), CSV export, print-as-PDF, and a public totals-only impact page.
Analytics	Pulse charts (Actual vs Estimate tags), community insights, badges and points that count only verified work, what-if city simulator for coordinators.
Notifications and offline	In-app bell, web push, email, SMS and WhatsApp (Twilio, with SAFE / NEED HELP / STATUS / SOS text commands). Installable PWA with offline queue for reports, SOS, confirmations and check-in replies.
Accessibility and language	English/Hindi toggle, voice input, keyboard-accessible map, skip link, text size, high contrast, reduced motion.

> **Trust and honesty principles the design must keep visible:** only humans verify; AI output is always labelled a suggestion; simulated data is always labelled "simulated" or "estimate"; the app states repeatedly that it is **not an emergency service**; location is opt-in and only a ~1 km cell or a user-picked pin is stored; every coordinator action is audit-logged.

2. Roles and what each can do

Role	Who	Key permissions (enforced on the server)
Resident	Default member	Report, merge/confirm, flag, withdraw, add updates, give "looks fixed" feedback, SOS, check-in reply, extra-help register, neighbor exchange, agent drafting and approval of own report.
Volunteer	Resident who opted in	Everything a resident can plus join missions, set availability, skills and abilities, share approximate area, post proof of fix, update SOS status.
Moderator	Set by a coordinator	Verify/dispute reports, hide/restore, approve or reject resolutions. Can see volunteer overlay.
Coordinator (admin)	Creator of a named neighborhood, or approved by one	All of the above plus advance status, create missions, assign volunteers, approve action plans, resources and deliveries, SOS acknowledge/assign, check-ins, notices, preparedness plan, authorities directory and hand-offs, escalations, What-if, Reports, People tab.
Platform admin	Emails in `PLATFORM_ADMINS`	Acts as coordinator in any neighborhood.

The demo has a Role dropdown in the header so all four roles can be previewed without an account.

3. Current visual language (reference for redesign)

Mood

Dark "mission control" interface: deep navy background with a radial glow, frosted-glass panels (blur, 1px translucent blue border), neon cyan/blue accents, soft glows on active items. Typeface **Sora** for UI and **JetBrains Mono** for timestamps.

Colour tokens

- bg `#04060c`
- ink `#e8f0ff`
- dim `#8b9bbd`
- cyan (focus, active) `#3de8ff`
- blue (primary) `#3b82ff`
- amber (in progress, warn) `#ff9a2e`
- red (open, critical, crisis) `#ff3b4e`
- green (resolved) `#2dff9a`

Status colour semantics (used on pins, dots, chips)

- **Red:** Reported, Under review
- **Amber:** Assigned, In progress, Resolution pending review
- **Green:** Resolved

Components in use

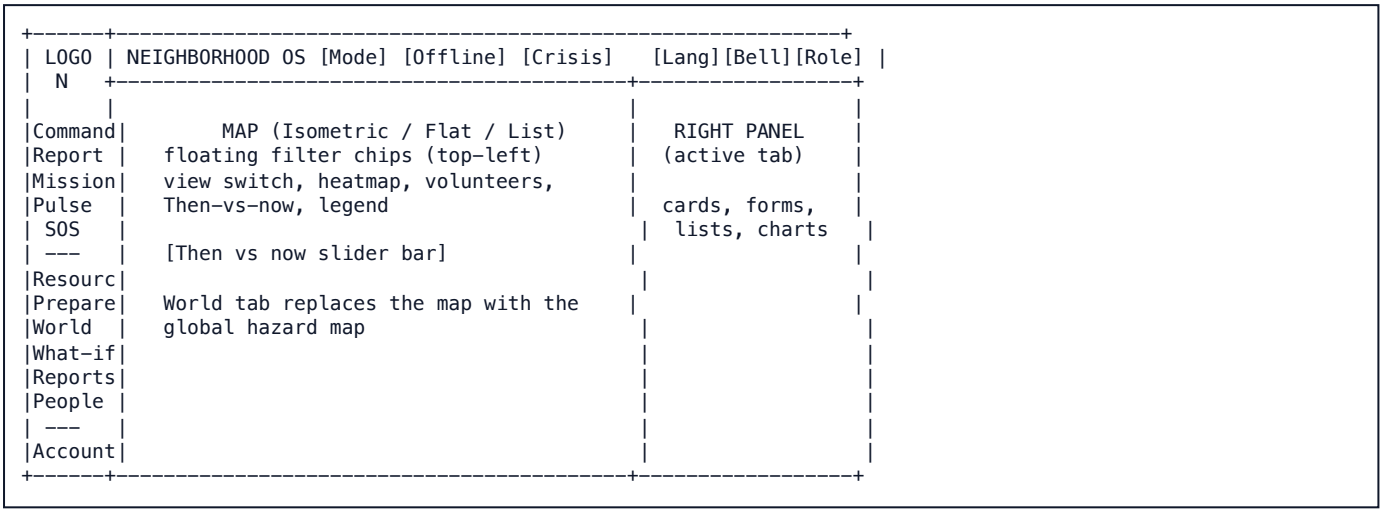
- Glass cards, 10-18px radius
- Primary "go" button: blue-to-cyan gradient, dark text
- Warn button: amber outline
- Chips: status, role, Actual/Estimate, Shortage, Done
- Issue row: colour dot, title, sub-line, timestamp
- Stats trio: Open / In progress / Fixed
- Progress bar (mission volunteers)
- Toasts (success and error), bell popover, modal-less side panel

Signature interactions

- Pulsing red "Crisis mode" chip
- Before/after drag slider
- Animated lines from volunteers to missions
- Panel cross-fade between tabs

4. App shell (after entering a neighborhood)

Single-page app. Desktop layout is a three-column grid: **left icon rail (76px)**, **centre map**, **right panel (420px)**, with a **top header bar** over centre and right. There is no URL routing between tabs; the active tab swaps the right panel (and the map layer for World). Mobile/tablet behaviour is not specified in the code and is a design opportunity.



Left rail navigation

Tab	Label	Icon	Visible to
`feed`	Command	Radar	All
`report`	Report	PlusCircle	All
`missions`	Missions	Users	All
`pulse`	Pulse	BarChart3	All
`sos`	SOS (red accent, prominent in Crisis Mode)	Siren	All
`resources`	Resources	Package	All
`prepare`	Prepare	ShieldCheck	All
`world`	World	Globe	All
`sim`	What-if	FlaskConical	Coordinator only
`reports`	Reports	FileText	Coordinator only
`people`	People	UserCog	Coordinator only, real mode only
`account`	Account	User	All (bottom of rail)

Header bar

- **Brand block:** "NEIGHBORHOOD OS" with sub-line "Simulated demo" or "Named / Area neighborhood" plus the neighborhood name.
- **Mode chip:** "Demo mode" or "Live · Role".
- **Connectivity chips:** "Offline" when disconnected or showing stale data; "N waiting to send" for the offline queue.
- **Crisis chip:** pulsing red, "Crisis mode: official alert" or "Crisis mode: N clusters".
- **Language select** (English / हिन्दी), **notifications bell** with unread badge and popover of the last 8 events addressed to the user, then in demo: Role dropdown, Reset demo, Exit demo; in real mode: Switch neighborhood.

Map overlays (centre)

- **Floating filter chips** for the six statuses (toggle), and a Resources toggle.

- **View switch:** Isometric, Flat, List. List replaces the map with a sortable issue list.
- **Heatmap** of open issues weighted by priority. **Volunteers** overlay (coordinators and moderators only). **Then vs now** button opens a bottom slider bar (0 to 60 days ago) that redraws the map from the activity log.
- **Picking mode:** on Report, Resources and SOS tabs the cursor becomes a crosshair and clicking drops a "Chosen location" pin.
- Loading state ("Loading neighborhood map…"), a note when OSM data is thin and a labelled schematic city is drawn instead, SIMULATION banner when the What-if overlay colours the map.
- Pins: issue pins coloured by status with a progress ring when the AI agent is tracking it; SOS pins; resource markers; crisis zones where reports cluster.

5. Page and panel inventory

Every item below is a distinct screen or panel to design. "API" lists the endpoints it relies on (see section 6). Most mutations go through one generic endpoint, `POST /api/n/:id/action`, with an action type shown as `t: "..."`.

5.1 Onboarding / sign-in gate

Shown when no neighborhood is active. Component `Onboarding.tsx`. Route: `/`. Invite links use `/?n=<neighborhood id>`.

- Brand headline "NEIGHBORHOOD OS" and tagline "See your neighborhood's problems on a live map, fix them together, and prove they are fixed."
- Primary button **"Try the simulated demo (no account)"**.
- **Auth card** with three modes: Sign in, Create account (display name with privacy note, email, password 8+), Forgot password. Inline success/error message. Notice when Supabase is not configured: "Real neighborhoods need an account service."
- **Signed-in state:** "Signed in as …", Sign out, **Your neighborhoods** list (up to 5).
- **Find a neighborhood:** "Use my location" (shares only the ~1 km cell), place search box (geocode results), result card showing the area, **Named neighborhoods nearby** list with member counts, and **Create a named neighborhood** form (name, radius 200 m to 2 km, "Create and become its coordinator").
- Footer disclaimer: not an emergency service.

API: `GET /api/me` · `GET /api/neighborhoods/mine` · `POST /api/neighborhoods/cell` · `POST /api/neighborhoods` · `POST /api/neighborhoods/:id/join` · `GET /api/neighborhoods/nearby` · `GET /api/neighborhoods/search` · `GET /api/geocode` (Supabase Auth handles sign-in itself)

5.2 Command tab (feed) - the home panel

Component `Feed` in `App.tsx`. Right panel, default tab.

- **Alert banner** (official hazards overlapping the area) linking to World tab.
- **Safety check-in card** (compact) when a check-in is open.
- **Notice board**: pinned and urgent notices; coordinators see a "Post a notice" form.
- **"Fixes waiting for review"** card listing issues in Resolution pending review.
- **Stats trio:** Open (red) / In progress (amber) / Fixed (green).
- **Demo-only card "Guided AI agent demo":** step X of N, sample Hindi report quote, "Run next step", "Simulate an earthquake alert" / "Clear simulated alert".
- **Empty state** for a new neighborhood: "This neighborhood is just getting started" with a "Report a problem" button.
- **Live issues list** sorted by priority score. Each row: status dot, title, sub-line "Status · Verification · score N · hidden", relative time.
- **Event log**: last 10 events with timestamps and tone colour.
- Selecting an issue opens the **Issue detail** (5.3) at the top of the panel.

API: `GET /api/n/:id/state` (polled every 5 s with `?v=version`) · `GET /api/n/:id/map` · action calls

5.3 Issue detail (drawer inside Command)

Component `Detail.tsx` (232 lines, the richest screen).

- **Header:** title, category, priority, status chip, verification chip (Unverified, Under review, Verified by coordinator, Disputed, Resolved), report count, hidden flag, author, age. Original photo.
- **Priority score:** 0-100 with a list of the reasons and points. Caption "A fixed set of rules adds these points. It only sorts the list."
- **Resident actions:** "I see this too", "Withdraw my report", "Report spam or abuse", "Does it look fixed?" with optional comment and the buttons "It looks fixed" / "Not fully fixed".
- **Coordinator/moderator actions:** Mark verified, Mark disputed, Advance status, Hide/Restore report, Create mission, Approve resolution, Request more work (reason required). Note "You asked for this review, so someone else must approve it."
- **AI agent panel** (see 5.4).
- **Action plan** (suggestion): steps, department, volunteers, effort, materials, safety notes. Buttons "Approve plan and create mission" and "Print action plan" (printable page).
- **Suggested volunteers** (rule-based) with assign buttons.
- **Proof of fix:** Before/After drag slider, "Upload evidence", progress note field, "Add an update or evidence". Timeline "Updates and evidence from neighbors". AI photo check (labelled AI) and AI before/after comparison.
- **Hand-off to an authority** (coordinator): choose authority, reference number, status tracker, overdue warning "No resolution after 7 days."
- **Assignment history**.

API: action types `verify, advance, merge, flag, withdraw, spam, hide, restore, mission, join, assign, proof, update, feedback, approveResolution, rejectResolution, actionPlanApprove, handoff, handoffStatus` · `POST /api/n/:id/photos` · `GET /api/photos/:pid` · `POST /api/n/:id/ai/compare` · `POST /api/n/:id/ai/image`

5.4 AI agent panel (inside Issue detail) and Analysis card (inside Report)

Components `AgentPanel.tsx`, `AnalysisCard`. Logic in `src/agent.ts`, route table in `src/routes.ts`.

- Intro: "The agent understands the report, picks the complaint route, drafts the complaint, and suggests a community mission. You approve every step."
- **Progress stepper:** Understood, Duplicates checked, Route, Draft, Approved, Sent, Reference received, Mission, Volunteers, Proof of fix, Verified resolved.
- **Understanding card:** detected language, category, priority, English summary, source label (AI vs rule-based), "Apply" button.
- **Route card:** authority, channel, what to include (checklist), deadline in days, escalation ladder. Warning "Check this route against your city's official website before sending."
- **Complaint draft:** subject and body, English/Hindi switch, editable, "Approve". After approval: copy text, "Mark as sent" (copied or emailed) with optional reference number. Caption "This app does not file on government portals for you."
- **Escalation:** next-level draft with a reference to the earlier complaint, shown when the deadline passes; coordinator button; cancelled message "The issue was resolved, so the escalation is not needed."
- **Mission suggestion:** cleanup for garbage; safe follow-up crew for water, lights, roads and hazards (never touching pipes, wires or open manholes).

API: `POST /api/n/:id/agent/analyze` · actions `agentStart, agentDraft, agentApprove, agentSend, agentRef, agentEscalate`

5.5 Report tab

Component `Report` in `App.tsx`. Map is in picking mode.

- Location line: "Click the map to choose the location" or coordinates once picked.
- Fields: Short title (120 chars, with voice button), Category select, Priority select, Description textarea (1000 chars, privacy hint "do not include names, phone numbers or faces", voice button), Photo picker (re-encoded in browser to strip GPS), photo thumbnail, AI photo check.
- Button **"Understand and prepare complaint"** showing the Analysis card.
- **Possible duplicates card:** likely category, nearby reports with % match and "Add to this report", "Get AI summary" (real mode), AI suggestion vs rule-based label.
- **Submit report** (primary). Footer: new reports stay Unverified; call emergency number for life-threatening situations.
- Validation toasts: choose location first, add a title.

API: action `report` (and `merge`) · `POST /api/n/:id/photos` · `POST /api/n/:id/agent/analyze` · `POST /api/n/:id/ai/suggest` · `POST /api/n/:id/ai/image`

5.6 Missions tab

- **New community mission** form (coordinator): title, volunteers needed (1-200).
- **My volunteer profile** (volunteer): Available/Unavailable toggle, skills as selectable chips (the 7 categories), emergency abilities chips (first aid, medical, driving, boat, tools, cooking, translation, counseling).
- **My approximate area** (volunteer): consent text, "Use the pin I chose on the map", "Use my current location", clear. Rounded to ~100 m, visible only to the volunteer and coordinators.
- **Mission cards:** title, Done chip, progress bar, "X of Y volunteers · N volunteer hours", "Join mission".
- Empty state: "No missions yet. A coordinator can create one." Resident hint: switch to volunteer in Account.

API: actions `mission, join, assign, availability, skills, abilities, volunteerArea, volunteerAreaClear`

5.7 Pulse tab (analytics)

- Date range: Last 7 / 30 / 90 days, all time. Summary numbers: Avg hours to fix, Missions done, Contributors.
- Charts (Recharts, dark theme): Reported vs resolved, Open issues over time, By category, Hours to resolve, Volunteer hours by mission. Each tagged **Actual** (green) or **Estimate** (amber).
- Recurring problem areas (hotspots; "a pattern in past reports, not a prediction"). Top contributors leaderboard ("Points count only verified reports and completed fixes").
- **Community intelligence** (AI insights, labelled). Demo note "All figures come from simulated demo data".
- Pointer to the map's "Then vs now" button.

API: `POST /api/n/:id/ai/insights` (everything else is computed client-side from the neighborhood document)

5.8 SOS tab

Component `SosPanel.tsx`. Highlighted red and promoted in Crisis Mode.

- Persistent banner: call your local emergency number first; this is not an emergency service.
- **Ask for help** form: need (medical, rescue, shelter, food and water, other), number of people, note (voice input), location chosen on the map or from the device (only sent on Send), explicit **consent checkbox**, Send.
- **Your requests** (resident) or **SOS board** (coordinator): cards with need, people, status (open, acknowledged, assigned, helped, closed), age, nearest shelter or aid post. Coordinator buttons: Acknowledge, "Assign a volunteer…" dropdown. Volunteer/author: Mark helped, Close, Cancel.
- Others see only a count. Empty state "No SOS requests right now." Privacy line: closing deletes location, note and contact.
- Limits: one active request per person, 3 per day, auto-wipe at 24 h.

API: actions `sos, sosAck, sosAssign, sosStatus, sosCancel` · inbound `POST /api/sms` (text "SOS medical 3 …")

5.9 Resources tab

- **Resource radar:** cards with name, kind, note, "Shortage" chip. Coordinator: delivery note + "Record delivery", "Flag shortage".
- **Add a resource** (coordinator): uses the map pin; name, kind (shelter, water, aid, donation, assembly, equipment), note.
- **Neighbor exchange:** offers and requests (category, title, details), claim, short message thread, mark done, remove. Safety copy: no phone numbers or addresses, be careful when meeting people.

API: actions `addResource, delivery, shortage, shareAdd, shareClaim, shareMsg, shareDone, shareRemove`

5.10 Prepare tab

- **Start / close safety check-in** (coordinator) with title. Members answer "I am safe" or "I need help"; coordinators see who has not replied, with registered neighbors listed first.
- **Preparedness plan:** Be ready, Evacuation routes and assembly points, Contacts. Coordinator edit form; empty state for no plan.
- **Assembly points and shared equipment** (pointer to Resources). **Volunteer abilities** summary of currently available volunteers.
- **Extra-help register:** optional, consent-based, coordinator-only visibility, short note without address, removable any time.

API: actions `checkinStart, checkinReply, checkinClose, planSet, careSet, careRemove`

5.11 World tab

Components `WorldMap` (replaces the centre map) and `WorldPanel` (right panel). Source `server/world.ts`, 10-minute server cache.

- Global map with event markers (earthquake, wildfire, storm, flood, volcano) sized/coloured by severity (info, watch, warning, severe) and the user's neighborhood marked.
- Panel: title "World layer", disclaimer "Official public hazard feeds, shown as published. This layer only informs; it never starts a rescue by itself."; filters by type and severity; event list with source (USGS, NASA EONET, GDACS, CAP, Simulated), magnitude, time and link; loading and "No events match these filters" states; source attribution footer.
- Alert logic: hazards overlapping the neighborhood (e.g. M6+ quake within 150 km, fire within 25 km) raise banners; warning/severe triggers Crisis Mode.

API: `GET /api/world` · alerts arrive inside `GET /api/n/:id/state`

5.12 What-if simulator (coordinator)

- Badge **SIMULATION**; caption that everything is an estimate and changes nothing.
- Controls: volunteers, team size, hours per day, assumed authority delay, "Which issue first?" ordering. Comparison against today's setup.
- Outputs: projected issues resolved chart, workload and resources (estimate), projected order of work list, optional map overlay colouring pins by projected completion. Assumptions paragraph.

API: none (client-side, `src/sim.ts`)

5.13 Reports tab (council reports, coordinator)

- Title "Council reports"; for coordinators, resident associations and local offices; exports contain no resident names.
- Filters: Last 7 / 30 / 90 days / all, category (All categories). Actions: Export CSV, Print / save as PDF, "Copy public summary link".
- Report body: "<Neighborhood>: report for the last N days", totals, avg hours to fix, Reports and fixes per week chart, By category, Recurring problem areas, **Passed to authorities (N)** hand-off tracker (acknowledged, overdue).
- **Authorities directory:** name, email, phone, add/remove.

API: actions `authoritySet, authorityRemove, handoff, handoffStatus` · public link uses `GET /api/public/:id/summary`

5.14 People tab (coordinator, real mode)

- Member list: name, "Wants to coordinate" chip, role dropdown (Resident, Volunteer, Moderator, Coordinator). A neighborhood cannot lose its last coordinator.
- **Coordinator activity log**: timestamp, who, action, detail (last 30). Empty state "Nothing recorded yet."

API: `GET /api/n/:id/members` · `POST /api/n/:id/members/:uid/role` · `GET /api/n/:id/audit`

5.15 Account tab

- **Identity card:** handle, email, role and neighborhood. "I want to volunteer" / "Stop volunteering", "Ask to become a coordinator", "Copy invite link". In demo: a "Simulated demo" card with "Create an account or sign in".
- **My neighborhoods:** switch between up to 5, "Find another neighborhood", "Leave this neighborhood" (confirm).
- **My impact:** badges and points (only verified work); empty "No badges yet."
- **Notification settings:** Push on this device, Email, SMS, WhatsApp (phone in international format, 6-digit code verification, remove), "Not set up on this server" states, SMS command help, connected CAP feed count.
- **Accessibility and language:** language, text size, high contrast, reduced motion, voice-input note.
- **Safety and privacy** card (five statements, see principles above).
- Sign out, **Delete my account** (confirm; name becomes "Former member").

API: `POST /api/n/:id/me/role` · `POST /api/n/:id/coordinator-request` · `POST /api/n/:id/leave` · `GET /api/capabilities` · `GET/POST/DELETE /api/n/:id/channels[/:kind]` · `DELETE /api/me`

5.16 Public impact page (no login)

Route `/?impact=<neighborhood id>`, component `PublicImpact.tsx`.

- Brand, neighborhood name, "Community-reported figures. No names or locations are shown."
- Four big numbers: Avg hours to fix, Volunteer hours, Missions done, Neighbors helped. Link "Open NEIGHBORHOOD OS".

API: `GET /api/public/:id/summary` (rate-limited, no auth)

5.17 Cross-cutting UI states to design

- **Toasts:** success (green) and error (red); queued-offline variant.
- **Crisis Mode:** red chip, official alert banner, emergency-services-first notice, prominent SOS tab, red crisis zones on the map.
- **Offline/stale:** "Offline" chip, "N waiting to send", cached last state.
- **Empty states** for every list; **loading** states for map, world feed and AI calls; **role-gated** hints ("To join missions, switch to volunteer").
- **Bell popover** with Escape and outside-click close; **skip link** "Skip to the list".
- **Printable views:** action plan and council report.
- **PWA:** installable (name "Neighborhood OS", theme/background `#04060c`, icons 192/512/SVG), service worker for offline and push.

6. API endpoints

Express server on port `8787` (proxied by Vite in development). All routes except health, public summary and the SMS webhook require a Supabase bearer token. Errors return `{ "error": "message" }`. Many routes are rate limited per user.

Method	Path	Auth	Purpose
GET	`/api/health`	none	Liveness check, returns `{ ok: true }`.
GET	`/api/me`	user	Current user: id, email, handle, platformAdmin.
DELETE	`/api/me`	user	Delete account; removes memberships and photos, author name becomes "Former member".
GET	`/api/capabilities`	user	Which optional services are configured: ai, email, sms, whatsapp, push, and CAP feed count.
Neighborhoods			
GET	`/api/neighborhoods/mine`	user	Neighborhoods the user belongs to, with role.
POST	`/api/neighborhoods/cell`	user	Join or auto-create the ~1 km area cell for a cell id. Body: `{ cell }`. 30/hour.
POST	`/api/neighborhoods`	user	Create a named neighborhood. Body: `{ name (3–60), lat, lng, r (200–2000 m) }`. Creator becomes coordinator. 3/day.
POST	`/api/neighborhoods/:id/join`	user	Join by id (named neighborhood or area cell).
GET	`/api/neighborhoods/nearby?lat&lng`	user	Up to 20 named neighborhoods near a point, with member counts.
GET	`/api/neighborhoods/search?q`	user	Name search, up to 10 results.
GET	`/api/geocode?q`	user	Place search through OpenStreetMap Nominatim. 20/min.
Neighborhood state and actions			
GET	`/api/n/:id/state?v=`	member	The neighborhood JSON document filtered for the caller's role, plus `role` and active `alerts`. If `v` equals the current version the server returns `{ unchanged: true }`. Polled every 5 s.
POST	`/api/n/:id/action`	member (role checked)	The single write endpoint. Body is an Action `{ t, ... }` (catalogue in 6.1). Returns `{ doc, result, version, role }`. Locks the row, writes the audit log for coordinator actions, triggers notifications afterward.
GET	`/api/n/:id/map`	member	Cached OSM roads, buildings, parks and water for the area (30-day cache). Returns `{ source: "none" }` when OSM data is too thin.
GET	`/api/world`	user	Global hazard events from USGS, NASA EONET and GDACS with sources and updated time (10-minute cache).
Photos			
POST	`/api/n/:id/photos`	member	Upload a base64 data-URL (JPEG/PNG/WebP, max 300 KB, validated by magic bytes). Returns `{ id }`. 30/day.
GET	`/api/photos/:pid`	member of its neighborhood	Stream the photo, private cache 1 h.

People and roles			
GET	`/api/n/:id/members`	coordinator	Member list with roles and coordinator-request flag.
POST	`/api/n/:id/members/:uid/role`	coordinator	Set a member's role. Refuses to remove the last coordinator. Audit-logged.
GET	`/api/n/:id/audit`	coordinator	Last 30 coordinator actions.
POST	`/api/n/:id/me/role`	member	Switch yourself between resident and volunteer.
POST	`/api/n/:id/coordinator-request`	member	Ask to become a coordinator (5/day).
POST	`/api/n/:id/leave`	member	Leave a neighborhood; last coordinator must promote someone first.
Notification channels			
GET	`/api/n/:id/channels`	member	The user's linked channels (push, email, sms, whatsapp) with masked targets and verified flag.
POST	`/api/n/:id/channels/:kind`	member	Link a channel. push: `{ subscription }`. email: no body. sms/whatsapp: `{ phone }` to send a 6-digit code, then `{ code }` to verify (10-minute expiry).
DELETE	`/api/n/:id/channels/:kind`	member	Unlink a channel.
POST	`/api/sms`	Twilio signature	Incoming SMS/WhatsApp webhook. Commands: SAFE, NEED HELP, STATUS, SOS <medical|rescue|shelter|food> <people> <text>. Replies in TwiML XML.
AI helpers (all labelled suggestions, rule-based fallback without a key)			
POST	`/api/n/:id/agent/analyze`	member	Body `{ text }`. Returns language, category, priority, English summary, source. 40/hour.
POST	`/api/n/:id/ai/suggest`	member	Body `{ title, desc, cat, lat, lng }`. Summary, likely needs, source. 30/hour.
POST	`/api/n/:id/ai/image`	member	Body `{ photo, mode, cat }`. Photo check for a report or a fix. 20/hour.
POST	`/api/n/:id/ai/compare`	member	Body `{ issueId, photo }`. Compares the original with an after photo. 20/hour.
POST	`/api/n/:id/ai/insights`	member	Community intelligence summary for Pulse. 10/hour.
Public			
GET	`/api/public/:id/summary`	none	Totals-only impact summary (name, avg hours to fix, volunteer hours, missions done, neighbors helped). 30/min per IP.
GET	`/* (non-API)`	none	In production the server serves the built single-page app and falls back to `index.html`.

6.1 Action catalogue (`POST /api/n/:id/action`)

Action	Payload	Allowed roles	Used by screen
`report`	title, desc, cat, priority, lat, lng, photo?, agent?	all	Report
`merge`	id	all	Report duplicates, "I see this too"
`flag`	id	resident	Issue detail
`withdraw / spam`	id	all	Issue detail

`verify`	id, v (verified | disputed ...)	coordinator, moderator	Issue detail
`advance`	id	coordinator	Issue detail
`hide / restore`	id	coordinator, moderator	Issue detail
`approveResolution / rejectResolution`	id, note? / reason	coordinator, moderator	Issue detail
`feedback`	id, ok, note?	all	"Does it look fixed?"
`update`	id, text, photo?	all	Issue detail timeline
`proof`	id, text, photo?	volunteer, coordinator	Proof of fix
`actionPlanApprove`	id	coordinator	Action plan
`mission`	title, need, issueId?	coordinator	Missions, Issue detail
`join`	mid	volunteer	Missions
`assign`	mid, name	coordinator	Issue detail, notifies the volunteer
`availability / skills / abilities`	available / skills[] / abilities[]	volunteer	Missions profile
`volunteerArea / volunteerAreaClear`	lat, lng, consent	volunteer	Missions, map overlay
`addResource`	name, kind, lat, lng, note	coordinator	Resources
`delivery / shortage`	id, note / id	coordinator	Resources
`shareAdd / shareClaim / shareMsg / shareDone / shareRemove`	kind, cat, title, note / id, text	all	Neighbor exchange
`sos`	need, people, note, lat, lng, consent	all	SOS
`sosAck / sosAssign`	id / id, name	coordinator	SOS board
`sosStatus`	id, status (helped | closed)	volunteer, coordinator	SOS board
`sosCancel`	id	all (author)	SOS
`checkinStart / checkinClose`	title	coordinator	Prepare
`checkinReply`	ok, note?	all	Command card, SMS
`careSet / careRemove`	need, note, consent	all	Prepare register
`planSet`	routes, notes, contacts	coordinator	Prepare
`noticeAdd / noticeRemove`	title, body, urgent, pinned / id	coordinator	Notice board
`authoritySet / authorityRemove`	name, email, phone / id	coordinator	Reports directory
`handoff / handoffStatus`	id, authorityId, ref / id, status, ref?	coordinator	Issue detail, Reports
`agentStart / agentDraft / agentApprove / agentSend / agentRef`	id + lang, original, summaryEn, source / subject, body / via (copied | emailed), ref	all	AI agent panel
`agentEscalate`	id	coordinator	AI agent panel
`demo / reset`	none	demo mode only (403 on server)	Demo controls

Rate limits (per user per neighborhood per day): report 10 (4/hour), proof 30, spam 20, merge 40, flag 20, mission 20, addResource 20, sos 3, update 40, feedback 30, agent steps 20-60, notices 20, check-ins 10. Coordinator actions are written to the audit log.

7. Data model (what screens can display)

Postgres tables (`database/schema.sql`)

Table	Columns and purpose
`neighborhoods`	id, kind (cell | named), name, center lat/lng, radius_m (200-2000), created_by, version, **doc** (jsonb, the whole neighborhood state), cached OSM **map** and fetch time.
`members`	neighborhood_id, user_id, display_name, role (resident | volunteer | moderator | admin), coordinator_requested_at, joined_at.
`photos`	id, neighborhood_id, user_id, mime, data (bytea). Photos live in Postgres.
`audit_log`	neighborhood_id, user_id, who, action, detail, at. Coordinator transparency log.
`channels`	user_id, neighborhood_id, kind (push | email | sms | whatsapp), target, verified, verification code.

The neighborhood document (`doc`), as the UI receives it

Collection	Content
`issues[]`	id, title, desc, cat, priority, status, verif, lat, lng, author, created, resolvedAt, reports (confirm count), hidden, photo, updates[], feedback, assignment history, plan, handoff {authorityId, ref, status, email}, agent state (analysis, route, draft, approvals, escalations).
`missions[]`	id, title, need, joined[], hours, done, issueId.
`volunteers[]`	name, available, skills[], abilities[], optional approximate area.
`resources[]`	id, name, kind, lat, lng, note, low (shortage), deliveries.
`sos[]`	need, people, note, lat/lng (role-restricted), status, assignee, created. Non-privileged viewers receive only a count.
`checkins / care register`	Open check-in with replies; extra-help register entries (coordinator-only).
`notices[], shares[], plan, authorities[]`	Notice board, neighbor exchange threads, preparedness plan, authority directory.
`events[]`	Event log (t, text, tone, optional `to` for targeted notifications). Also used to rebuild the "Then vs now" map.
`hazard alerts`	Computed server-side per neighborhood: type, severity, distance, radius, source, instructions.

Enumerations

- **Status:** reported, review, assigned, progress, pending (Resolution pending review), resolved
- **Verification:** unverified, review, verified, disputed, resolved
- **Category:** pothole, garbage, streetlight, water, unsafe, blocked, other
- **Priority:** low, medium, high, critical
- **Resource kind:** shelter, water, aid, donation, assembly, equipment
- **SOS need:** medical, rescue, shelter, food_water, other. **SOS status:** open, acknowledged, assigned, helped, closed
- **Hazard severity:** info, watch, warning, severe. **Hazard types:** earthquake, wildfire, storm, flood, volcano
- **Abilities:** first_aid, medical, driving, boat, tools, cooking, translation, counseling

8. Key user flows to prototype

#	Flow	Steps
1	First visit to demo	Onboarding, "Try the simulated demo", Command tab with guided agent card, switch Role in header, "Run next step" plays the Hindi-report agent flow.
2	Join a neighborhood	Sign up, "Use my location" or search, choose area / named neighborhood nearby / create one, land on Command. Invite link `/?n=id` skips search.
3	Report a problem	Report tab, click map, type or speak, pick category and priority, add photo, "Understand and prepare complaint", review duplicates, Submit, toast, jump to the new issue in Command.
4	Coordinator triage to fix	Open issue, Mark verified, approve action plan (creates mission), assign volunteers, volunteers post proof, coordinator asks for review, moderator approves, before/after slider, pin turns green.
5	Complaint to authority	Agent panel: route, Hindi/English draft, edit, approve, copy, mark sent with reference number, deadline passes, escalation draft, approve.
6	Crisis escalation	World tab shows a hazard, overlap raises banner, warning triggers Crisis Mode (chip, banner, SOS tab emphasis), coordinator starts a check-in, members reply safe / need help, SOS requests arrive on the board, coordinator assigns volunteers.
7	Ask for help (SOS)	SOS tab, read emergency-first notice, choose need and people, place pin, tick consent, Send, see status progress, request closes and location is wiped.
8	Report to the council	Reports tab, choose range, view charts and hotspots, hand off issues to authorities, export CSV or print PDF, share public impact link.
9	Plan with What-if	What-if tab, change volunteers and hours, compare projections, optional map overlay, nothing changes in real reports.
10	Link SMS or WhatsApp	Account, Notifications, enter phone, receive code, verify, later text SAFE / STATUS / SOS to the service.

9. Suggested brief for Claude Design

> **Design the NEIGHBORHOOD OS web app.** It is a civic platform with a live isometric neighborhood map at the centre, a 76px icon rail on the left and a 420px contextual panel on the right, topped by a status header. Keep the dark mission-control mood (deep navy, frosted glass, cyan/blue accents; red = open or critical, amber = in progress, green = resolved), but improve hierarchy, spacing and readability. Design all screens in section 5 for desktop and add tablet and phone layouts (bottom tab bar with a "More" sheet for the secondary tabs). Include light and dark themes if feasible. Show every role-gated variant (resident, volunteer, moderator, coordinator), plus empty, loading, offline and Crisis Mode states.
>
> **Must stay visible in the UI:** the "not an emergency service, call your local emergency number first" message on SOS and Account; AI output labelled as a suggestion; simulated data labelled; Actual vs Estimate tags on charts; consent wording on location and SOS; human-approval gates (verify, approve resolution, approve complaint). Support English and Hindi (Devanagari) text lengths and keyboard / screen-reader use.

Priority order if time is limited

1. Onboarding gate and app shell (header, rail, panel)
2. Map with overlays and issue pins
3. Command feed and Issue detail with proof-of-fix slider
4. Report form
5. SOS and Crisis Mode

6. Missions, Resources, Prepare
7. Pulse, Reports, What-if
8. World, People, Account
9. Public impact page

> **Known limits worth reflecting in the design copy** (from the README "Honest limits"): location claims are self-asserted; OpenStreetMap coverage varies (a labelled schematic city is drawn when data is thin); updates arrive by 5-second polling; SOS needs an internet connection; Hindi translations have not had native review; the authority names and escalation steps in the route table are generic and must be checked per city.